

Tuon DB Schema Review — Performance & Integrity

Reviewed file: tuon-db-schema.sql (11 tables, 170 lines)

Reviewed against: Tuon-Requirements.pdf (FR 1.1 Scanner, FR 2.1 Result Processing, FR 3.1 Exam Analytics, FR 4.1 Feedback, FR 5.1 Notification Alert)

Date: August 24, 2026

Verdict: The core entity model is sound, but there are **6 blocking defects**, several integrity gaps that will silently corrupt analytics, and essentially **no index coverage on foreign keys** — which directly threatens the stated non-functional targets ("process exam result within 30 seconds", "handle multiple concurrent scans and result updates").

Severity summary

#	Severity	Issue	Impact
1	Blocking	exams.created_by is NOT NULL with ON DELETE SET NULL	Runtime error, unsatisfiable constraint
2	Blocking	otp_flows.email is UNIQUE	Second OTP request per email fails forever
3	Blocking	otp_flows.user_id has DEFAULT gen_random_uuid() on an FK column	FK violation on any insert omits it
4	Blocking	No uniqueness on score_results (exam_id, student_id)	Duplicate scores per student in analytics double-count
5	Blocking	No uniqueness on answer_keys (exam_id, version)	Ambiguous answer key making auto-checking
6	Blocking	FR 5.1 (notifications) has no schema support at all	Requirement unimplemented
7	High	~13 foreign keys with no index	Slow joins, slow CASCADE deletes, concurrency contention
8	High	email stored in three tables with three separate unique constraints	Guaranteed data drift
9	High	Case-sensitive email uniqueness	Duplicate accounts (A@a.com vs a@x.com)
10	High	Per-topic scores buried in jsonb	FR 3.1 analytics cannot be queried or aggregated in SQL
11	High	Wide ON DELETE CASCADE chains from institutions	One row delete wipes the dataset
12	High	profiles.role is unconstrained varchar	No role integrity; no student role enforcement
13	Medium	Cross-institution / cross-course integrity not enforced	Student of Institution A submits a paper for Institution B's course

#	Severity	Issue	Impact
14	Medium	updated_at triggers missing on profiles and answer_keys	Stale audit fields
15	Medium	Duplicate index on refresh_tokens.token_hash	Wasted writes and storage
16	Medium	Numeric/typing problems (total_items numeric, unbounded varchar)	Bad data, wasted space
17	Medium	No RLS policies defined	Violates "only authorized users can access their own reports"
18	Low	Naming inconsistencies (email_created_at, message_at, encrypted_password)	Maintainability

Blocking defects

1. exams.created_by — impossible constraint

```
created_by uuid not null,
constraint exam_created_by_fkey foreign KEY (created_by)
references profiles (user_id) on delete set null
```

The column is NOT NULL but the FK action is SET NULL. Deleting an instructor's profile makes Postgres attempt to write NULL into a NOT NULL column, which raises an error and aborts the delete. The declared intent (preserve the exam, forget the author) never happens.

Fix: decide which you want.

- Preserve exam history (recommended): ON DELETE RESTRICT and keep NOT NULL.
- Allow orphaned exams: make the column nullable and keep SET NULL.

2. otp_flows.email UNIQUE breaks OTP re-issuance

```
constraint otp_flows_email_key unique (email)
```

An OTP flow is an **event**, not an identity. With this constraint, a user who never receives the first code can never be issued a second one — the insert fails with a unique violation, and the table can never hold OTP history for auditing or rate-limit analysis.

Fix: drop the unique constraint. If you want to enforce "at most one live OTP per user", use a partial unique index instead, which allows unlimited expired/consumed rows:

```
create unique index otp_flows_one_active_per_user
on public.otp_flows (user_id) where (is_active);
```

3. otp_flows.user_id default gen_random_uuid()

```
user_id uuid not null default gen_random_uuid (),
```

A foreign key column must never have a random default. Any insert that omits `user_id` fabricates a UUID that cannot exist in `users`, so it fails the FK check — a confusing error instead of a clear "user_id required". Drop the default.

4. score_results allows duplicate results per student per exam

The primary key is a surrogate `score_result_id`, and there is no unique constraint on `(exam_id, student_id)`. FR 1.1/2.1 involve smartphone scanning, where **rescans and retries are the norm** (paper jams, bad lighting, double-submits). Every rescan inserts a second row, and FR 3.1 analytics (AVG, COUNT, passing rate) then silently double-count that student.

Fix — pick one model and enforce it:

- One authoritative result per student per exam: unique `(exam_id, student_id)`, and have the scanner `INSERT ... ON CONFLICT DO UPDATE`.
- Keep attempt history: add `attempt_no` `smallint not null default 1` with unique `(exam_id, student_id, attempt_no)` plus a partial unique index on `(exam_id, student_id)` where `is_current`, and make analytics read only current rows.

The same problem exists on `exam_papers` `(exam_id, student_id)`.

5. answer_keys versioning is not enforced

`version` `smallint not null` exists but nothing prevents two rows with `(exam_id, version) = (... , 1)`, and nothing marks which version is authoritative. FR 2.1 compares scanned answers against "the stored answer keys" — with two candidate keys the scoring result is non-deterministic, which is a correctness bug in the product's core feature.

Fix:

```
alter table public.answer_keys
  add constraint answer_keys_exam_version_key unique (exam_id, version),
  add column is_active boolean not null default true;

create unique index answer_keys_one_active_per_exam
  on public.answer_keys (exam_id) where (is_active);
```

6. FR 5.1 (notification alerts) and result release have no schema

The requirements state the system "automatically notify student once the exam score released" and that notification fires "after an instructor finalizes exam results". The schema has:

- no `notifications` table,
- no `status / released_at / finalized_by` on exams,

- no delivery state (queued / sent / failed) for the email notification.

Without a release flag there is also no way to hide provisional scores from students, so partially-scanned results leak. Without a notification row there is no idempotency, so a retry sends duplicate emails.

Fix: add `exams.status` (draft / scanning / released), `released_at`, and `released_by`, plus a notifications table keyed on (`user_id`, `exam_id`, `kind`) for once-only delivery.

Performance issues

7. Foreign keys are almost entirely unindexed

Postgres creates an index for PRIMARY KEY and UNIQUE but **not** for foreign key columns — the referencing side must be indexed manually ([PostgreSQL constraints documentation](#), and see the [missing FK index anti-pattern](#)). Only `refresh_tokens` has any.

Unindexed FKs are missing on:

Table	Column(s)
<code>otp_flows</code>	<code>user_id</code> , <code>expires_at</code>
<code>profiles</code>	<code>institution_id</code>
<code>courses</code>	<code>institution_id</code>
<code>course_enrollment</code>	<code>course_id</code> (PK is (<code>user_id</code> , <code>course_id</code>), so course-first lookups scan)
<code>exams</code>	<code>course_id</code> , <code>created_by</code>
<code>exam_papers</code>	<code>exam_id</code> , <code>student_id</code>
<code>answer_keys</code>	<code>exam_id</code>
<code>score_results</code>	<code>exam_id</code> , <code>student_id</code>
<code>feedbacks</code>	<code>exam_id</code> , <code>student_id</code>

This hurts in two ways that map directly to your non-functional requirements:

1. **Read ath.** "Topic-wise score reports" and the instructor analytics screen both filter by `exam_id` or `student_id`. Every one of those is a sequential scan today. It looks fine on 50 rows and falls apart at 50,000.
2. **Write/delete ath.** This is the bigger risk. Every `ON DELETE CASCADE` forces Postgres to find matching child rows. With no index it sequentially scans the child table **and holds row locks while doing it**. Deleting one course scans `exams`, `exam_papers`, `answer_keys`, `score_results`, and `feedbacks` end to end — which will stall the "multiple concurrent scans" workload.

8-9. Email is duplicated and case-sensitive

email appears in `users`, `otp_flows`, and `profiles`, each with its own `UNIQUE`. There is no mechanism keeping them in sync, so an email change updates one and leaves two stale. Worse, the constraints are mutually inconsistent: `profiles` enforces `length(email) <= 50` while `users.email` is unbounded character varying — so a 60-character email registers successfully into `users` and then fails on profile creation, leaving an orphaned auth row.

Additionally, Postgres varchar comparison is case-sensitive, so `Jhon@vsu.edu.ph` and `jhon@vsu.edu.ph` are two separate accounts under `users_email_key`.

Fix: make `users.email` the single source of truth; drop email from `profiles` and `otp_flows` (join instead, or expose a view). Normalize case with either `citext` or a functional unique index, and move the length limit up to `users`:

```
create unique index users_email_lower_key on public.users (lower(email));
alter table public.users add constraint users_email_len_check check (length(email) <=
```

254 is the practical maximum address length; 50 will reject legitimate institutional addresses.

10. Per-topic analytics are trapped in JSONB

`score_results.scores jsonb`, `exam_papers.actual_answers jsonb`, and `exams.topics character varying[]` mean FR 3.1 ("generates topic-based performance analytics... allow filtering of reports by exam topic") must be computed by unnesting JSON at query time. Consequences:

- No index can serve "average score for topic X across a course" — every report is a full scan plus JSON parse.
- No constraint validates the JSON shape, so one malformed scanner payload silently corrupts a report instead of failing at write time.
- Topic names live in three places (`exams.topics`, the answer-key JSON, and the score JSON) with no referential link, so a typo in one breaks the join and the topic silently disappears from analytics.

Fix (recommended): normalize the analytics surface, which is the part you actually query.

```
create table public.topics (
  topic_id uuid primary key default gen_random_uuid(),
  institution_id uuid not null references public.institutions(institution_id) on delete cascade,
  topic_name text not null,
  constraint topics_name_per_institution_key unique (institution_id, lower(topic_name))
);

create table public.exam_topics (
  exam_id uuid not null references public.exams(exam_id) on delete cascade,
  topic_id uuid not null references public.topics(topic_id) on delete restrict,
  item_count integer not null check (item_count > 0),
  primary key (exam_id, topic_id)
);
```

```
create table public.score_topic_results (
  score_result_id uuid not null references public.score_results(score_result_id) on delete cascade,
  topic_id uuid not null references public.topics(topic_id) on delete restrict,
  correct_count integer not null check (correct_count >= 0),
  item_count integer not null check (item_count > 0),
  primary key (score_result_id, topic_id),
  constraint score_topic_range_check check (correct_count <= item_count)
);
create index idx_score_topic_results_topic on public.score_topic_results (topic_id);
```

Keep the raw jsonb as the immutable scan payload for audit and re-processing — that is a good use of JSONB. Just don't make it the query surface.

Minimum fix if you keep JSONB: add GIN indexes and shape validation.

```
create index idx_score_results_scores_gin on public.score_results using gin (scores jsonb);
create index idx_exams_topics_gin on public.exams using gin (topics);
alter table public.score_results add constraint scores_is_object check (jsonb_typeof(scores) = 'object');
```

15. Redundant index on refresh_tokens.token_hash

```
constraint refresh_tokens_token_hash_key unique (token_hash),
...
create index idx_refresh_tokens_token_hash on public.refresh_tokens using btree (token_hash);
```

The UNIQUE constraint already builds a btree index on that exact column. The second index is pure overhead — extra work on every token insert, extra storage, extra vacuum. Drop `idx_refresh_tokens_token_hash`.

Also worth adding, since token validation always filters on live tokens:

```
create index idx_refresh_tokens_active on public.refresh_tokens (user_id) where (revoked = false);
```

Integrity issues

11. Cascade blast radius is dangerous

`institutions → courses` is ON DELETE CASCADE, and `courses → exams → {exam_papers, answer_keys, score_results, feedbacks}` all cascade too. Deleting a single institution row silently destroys every exam, every scanned paper, and every historical score in the system. There is no soft delete and no audit trail to recover from.

Note the inconsistency: `profiles → institutions` is correctly RESTRICT, but `courses → institutions` is CASCADE. Same parent, opposite policy.

Fix: RESTRICT on the high-value parents (`institutions`, `exams`), and add `deleted_at timestamptz` soft deletes for anything an instructor can delete from the UI. Reserve CASCADE for genuinely

subordinate rows (`refresh_tokens`, `otp_flows`, `score_topic_results`).

The same reasoning applies to users: deleting a user currently cascades away their `exam_papers` and `score_results`, so a graduating student's departure erases the cohort's historical passing rate. Prefer `RESTRICT` plus anonymization.

12. `profiles.role` has no domain

```
role character varying not null,
```

Nothing stops `'student'`, `'Student'`, `'STUDENT'`, or `'stdent'`. Every authorization check in the backend is a string comparison against unvalidated data — a typo on insert becomes a silent privilege bug.

Beyond that, no constraint ties role to usage: nothing prevents an instructor from appearing as `exam_papers.student_id`, or a student from appearing as `exams.created_by`.

Fix:

```
create type user_role as enum ('student', 'instructor', 'admin');
-- or, less invasive:
alter table public.profiles
    add constraint profiles_role_check check (role in ('student','instructor','admin'));
```

Enforce role-appropriate references with a trigger or a `CHECK` on a denormalized role column if this matters for compliance.

13. Cross-institution leakage is possible

`exam_papers.student_id` references `profiles(user_id)` with no verification that the student belongs to the same institution as the exam's course. Likewise, `course_enrollment` lets a student enroll in any institution's course. For a multi-tenant review center this is a data-isolation defect, not just a modelling nit.

Fix: carry `institution_id` down onto `courses`, `exams`, and `score_results` and enforce composite foreign keys, or validate with a trigger on insert. Composite FKs are the stronger guarantee because the database enforces them even under concurrency.

Also note `course_enrollment.user_id` references `profiles` while `refresh_tokens.user_id` references `users`. Both work, but pick one convention — mixed parents make cascade reasoning error-prone.

14. `updated_at` is a lie on two tables

Only `courses` has `set_courses_updated_at_trigger`. `profiles.updated_at` and `answer_keys.updated_at` both default to `now()` and then **never change**, so any code that syncs on "modified since" will miss updates. Since `set_updated_at()` already exists, this is a two-line fix:

```

create trigger set_profiles_updated_at_trigger before update on public.profiles
  for each row execute function set_updated_at();
create trigger set_answer_keys_updated_at_trigger before update on public.answer_keys
  for each row execute function set_updated_at();

```

16. Type and constraint problems

Column	Problem	Recommendation
exams.total_items numeric	Allows 10.5 items; numeric is variable-width and slower to aggregate	integer not null check (total_items > 0)
exams.passing_rate numeric	No precision, no range check; name is ambiguous (percentage? threshold?)	numeric(5,2) check (passing_rate between 0 and 100), rename to passing_score_pct
otp_flows.attempts	No cap, so lockout logic lives only in app code	check (attempts between 0 and 5)
otp_flows	No created_at; can't audit or rate-limit	add created_at timestampz not null default now()
otp_flows.institution_name text	Denormalized copy of institutions.institution_name	replace with institution_id uuid references institutions
institutions.institution_name	No unique constraint	unique (lower(institution_name))
courses.course_name	No unique per institution	unique (institution_id, lower(course_name))
users	No created_at / updated_at at all	add both
all character varying	No length, mixed with text elsewhere	standardize on text + explicit CHECK
expires_at columns	No sanity check	check (expires_at > created_at)

17. No row-level security

Every table is in public with no policies. FR 3.1 explicitly requires "only authorized users can access their own reports". If this is a Supabase project — and `gen_random_uuid()`, `TABLESPACE pg_default`, and the public schema strongly suggest it — a table in public without RLS is readable by anyone holding the anon key ([Supabase RLS docs](#)). Note that Supabase is phasing out automatic Data API exposure for new public tables, but grants and RLS are separate layers — you still need explicit policies.

At minimum: enable row level security on all eleven tables, then policies so a student sees only rows where `student_id = auth.uid()`, and an instructor sees only rows within their `institution_id`. Write the institution check as a security definer helper function rather than a subquery in every policy, so it stays fast.

18. Naming and semantics

- `users.email_created_at` — almost certainly meant to be `email_confirmed_at` (or the table's missing `created_at`). As written its meaning is unrecoverable.
- `users.encrypted_password` — it's a hash, not ciphertext. Rename to `password_hash` so nobody tries to decrypt it.
- `feedbacks.message_at` and `score_results.scanned_at` are creation timestamps named inconsistently with the rest of the schema's `created_at`.
- `feedbacks` has no author column, so FR 4.1's "feedback per student per exam" can't record which instructor wrote it, and no `read_at` for the notification flow.
- `exam_papers` stores no scan image reference, no `scanned_by`, and no processing status, so a failed scan can't be retried or diagnosed — and concurrent duplicate submits have no idempotency key to dedupe on.
- Mixed `ON UPDATE CASCADE` vs. omitted: harmless because UUID PKs never change, but the inconsistency invites wrong assumptions.

Recommended order of work

1. **Correctness first (blocking, ~1 hour):** fix `exams.created_by`, drop `otp_flows.email` unique and its `gen_random_uuid()` default, add uniqueness to `score_results / exam_papers / answer_keys`.
2. **Indexes (low risk, high payoff):** create every missing FK index with `CREATE INDEX CONCURRENTLY`; drop the duplicate `refresh_tokens` index.
3. **Security:** enable RLS with policies before any real student data lands.
4. **De-duplicate email**, normalize case, and reconcile the length constraints.
5. **Add the release/notification model** so FR 5.1 becomes implementable.
6. **Normalize to ics and er-to ics scores** — do this before the analytics screens are built, because retrofitting it later means migrating live JSONB.
7. **Tighten cascades**, add `role` domain, add missing `updated_at` triggers, fix numeric types.

A ready-to-review migration covering steps 1–4 and 7 is in `tuon-db-schema-fixes.sql`. Run it against a copy first; the destructive statements are commented and flagged.